

URL shortener: features implemented

Service, scale-out, identity and observability, and the agentic SDLC orchestrator that delivered part of it.

At a glance	
Service	URL shortener (FastAPI): core API, safety, privacy-first analytics, abuse protection, identity, operations
Orchestrator	Governed agentic SDLC engine with 12 agents; delivered three of the service's features
Scale-out (optional)	PostgreSQL, Redis cache, Bloom filter, shared rate limits, event-driven analytics
Quality	552 tests, 100% line and branch coverage, 8 CI checks, load and chaos evidence
Versions	v0.9.0 built commit by commit; v0.10.0 and v0.11.0 delivered by orchestrator scenarios

1. URL shortener service

Core API

Feature	What it does
Create links	POST /api/v1/links; idempotent per owner and URL (safe retries return 200 with the same link)
Redirect	GET /{code}: 307 with Cache-Control: no-store, so every click reaches the service and is counted
Details and takedown	Link details; soft-delete takedown that keeps history and never reuses the code
Short codes	Random 7-character base62 from a cryptographic generator: not guessable, reveals no volume
Custom aliases	Memorable codes; duplicates are 409, reserved and invalid aliases are 400
Expiry	ttl_seconds; expired links return 410
Click caps	max_clicks enforced atomically: 40 concurrent redirects on a 10-click link give exactly 10; then 410
Errors	One JSON error format with a request ID everywhere; unknown fields rejected

Safety

Feature	What it does
SSRF protection	Rejects private, loopback and link-local IPs, .internal hosts, credentials in URLs, non-http(s) schemes
Redirect chains	Other shorteners (and the service's own host) are rejected
Look-alike domains	Mixed-script hostnames rejected, including punycode (e.g. paypal written with a Cyrillic letter)
Security headers	nosniff, Referrer-Policy, X-Robots-Tag, CSP, frame protection; HSTS over https
CORS	Off by default; exact origins only; no credentials
Trusted proxies	X-Forwarded-For read right to left, only from configured proxies: client IPs cannot be spoofed

Privacy-first analytics

Feature	What it does
Statistics	Clicks by day and by hour (UTC), top referrer domains, browser families, last click
Bots	Crawlers, link previews and scripts counted separately, never in totals
No raw IPs	Visitors counted with a keyed hash under a key that rotates daily; referrers keep only the domain
Private stats	Require the per-link token, the owner's API key, or an admin key; unknown codes also return 401

Abuse protection

Feature	What it does
Rate limiting	GCRA limits on creates and redirects; RateLimit-* and Retry-After headers
Bounded memory	Idle clients dropped first; an attacker cannot reset another client's limit

Identity and security

Feature	What it does
API keys	Owner and admin keys: hashed, revocable, shown once; constant-time comparison
Owner rights	Owners create links as themselves, read and take down only their own links
Audit identity	Every change recorded with the caller (owner:team-a, admin:alice) in a hash-chained log
Audit checkpoints	Signed checkpoints detect truncation and full rewrites that the hash chain alone cannot
Secrets	From environment or files (Vault Agent, Docker and Kubernetes secrets); never from the config file
Production profile	Refuses unsafe settings at startup and lists every problem at once (one JSON line, exit 2)

Operations and observability

Feature	What it does
Logging	Non-blocking JSON logs with redaction, request IDs and trace IDs; sampling of successful requests
Metrics	Prometheus /metrics by route template (bounded cardinality); token-protected in production
Tracing	OpenTelemetry spans per request, continuing incoming traceparent; OTLP export; trace ID in every log line
Dashboards and alerts	Grafana dashboard and Prometheus alert rules, checked against the real metric names
Health	Liveness and readiness probes; non-root container with health check

Optional scale-out (opt-in by configuration)

Feature	What it does
PostgreSQL	Same contract as SQLite; reversible Alembic migrations kept at the same schema version
Redis	Link cache with immediate takedown invalidation, Bloom filter for unknown codes, shared rate limits
Event-driven analytics	Transactional outbox, Redis Streams, idempotent aggregator, dead-letter queue, replay
Resilience	Circuit breaker and fail-fast 503s for Postgres; Redis outages fall back to the database; throttled failure logs

2. Agentic SDLC orchestrator

Agents do the work; the engine decides whether it passes. An agent cannot approve itself, skip a gate, bypass a policy or modify its own guardrails.

Governance engine

Capability	What it does
Stage graph	Dependencies, entry and exit gates, parallel stages with synchronisation
Recovery	Bounded retries with feedback, fallback agents, git-snapshot rollback, rework loops
Re-planning	Human amendments re-plan upstream; agents can insert stages (e.g. a migration review); stale results of reset stages are discarded
Human control	Approve, reject, amend or pause at checkpoints; a missing decision is never an approval; pause and resume; safe-stop
Policy guardrails	Security (formatted SQL, dangerous calls, secrets), compliance (personal data in logs, stories without criteria), change control (protected paths, scope, test deletion)
Accountability	Hash-chained audit log, artifact lineage, metrics: success rate, retry and rollback rates, MTTR

Agents and CLI

Item	Details
12 agents	Requirements (plus an LLM-backed version validated against a catalog, with deterministic fallback), impact analysis, design, migration, development, review, QA, security, docs (plus fallback), release
CLI	run, resume, stop, verify, and a human-gated promote into a repository

Scenarios (run in CI)

Scenario	What it demonstrates
Greenfield	A governed build of the whole service; recovery from an injected QA fault
Brownfield	Delivered click caps (v0.10.0): blocked an injected SQL flaw, sent a fail-open bug back, inserted a DBA migration review
Ambiguous	Delivered look-alike detection and clicks by hour (v0.11.0) after clarification and a mid-run re-plan
Ambiguous (LLM)	The same flow with the AI-backed requirements agent (labelled offline fixture in CI)

3. Quality and evidence

Area	Result
Tests	552 tests; 100% line and branch coverage for the service, the orchestrator and the infrastructure adapters
CI	8 checks: lint, strict typing, security scans, tests on Python 3.11 and 3.12, Docker integration, infrastructure, SDLC scenarios
Load tests	Three configurations, zero errors under sustained load
Chaos drills	Redis and Postgres outages during live traffic; two real problems found, fixed and re-measured
Documents	Requirements with traceability (32/32 criteria), design with 16 decision records and a threat model, code review report, QA report, AI usage ledgers, final summary

Known limits (stated in the documentation)

- No real LLM responses are recorded; CI uses a clearly labelled hand-written fixture.
- NFR-1 (redirect p99 under 50 ms at 500 requests/s) is not demonstrated: the load tests ran on a single CPU core.
- The greenfield scenario replays a reviewed baseline: it demonstrates governance, not code generation.